The Relay: cited answers from three papers under real rate limits

Code: github.com/Dishan18/Vaani (also in this folder). Raw data: results.csv (one row per request), data/probe/probe_raw.jsonl (every probe call with its headers and 429 body), data/results/detail.jsonl (per-step timings), data/experiments/ (tuning runs). My prompts to the assistant are in prompts.md.

1 What I built and why

The Relay answers questions about three papers and cites every claim. qwen3.8-27b reads the question, picks the papers that matter and rewrites the query. BM25 search pulls section-aware pieces, and searches again if the evidence looks thin. If it's still too weak, the pipeline refuses without calling a model. Otherwise gpt-oss-120b drafts the answer with every claim cited as [S#], a deterministic check makes sure the citations hold up, and a gpt-oss-20b verifier reads it too when there's spare capacity. Each Groq model has its own limits, so three models give roughly three times the capacity.

One idea shaped everything else: don't make anyone wait for budget that isn't coming. An answer costs about 1,700 tokens and each model allows 8,000 tokens a minute. Three models write about 10 full answers a minute; past that, something has to give. So before every model call, the client checks its own copy of Groq's buckets (synced from headers and 429 bodies) and predicts the wait, counting the queue ahead. If the drafting model is already backed up, the request goes lean: draft only, 4 pieces, about half the tokens. If the preferred model won't be free within 2 s, another model takes the call. If nothing frees up within 3 s, the person gets the cited passages straight away with no model call. Only an internal failure gives "busy, retry in N s".

2 The real service and my stand-in

Before designing anything I probed Groq with 119 requests (about 98K tokens): headers, an RPM wall, a TPM wall, concurrency bursts, paced latency (docs/probe_plan.md).

Limit	Value you found	How you found it	Confidence
Requests per minute	30 per model	30 tiny calls to gpt-oss-20b in 4 s; call 31 got 429 "requests per minute (RPM): Limit 30, Used 30"	high
Tokens per minute	8,000 per model; qwen also has 7,000 input tokens/min (ITPM)	`x-ratelimit-limit-tokens: 8000` on every reply; 429 bodies "TPM: Limit 8000" and, for qwen only, "ITPM: Limit 7000" (no header shows it)	high
Requests per day	1,000 per model, refilled continuously at 1 request per 86.4 s	`x-ratelimit-limit-requests: 1000`; `reset-requests / requests used` = 86.40 s on all 119 probe calls. Not hit on purpose	high (value), medium (behaviour at zero)
Tokens per day	200,000 per model (docs)	Groq docs table only; no header reports it, and no run (probe, Table 3, real loads) ever got a daily-limit 429	low
Requests in flight at once	no limit seen up to 8	bursts of 1/4/8/16 to qwen: 8 at once all OK with no slowdown; at 16 the ITPM limit hit first, so higher is unknown	low
What it returns when a limit is hit	HTTP 429, JSON `error.type` requests or tokens, `code` rate_limit_exceeded, message with Limit / Used / Requested; `retry-after` in seconds sized to this request; an RPM 429 has no token headers	raw 429 responses in data/probe/probe_raw.jsonl	high
Time until it recovers	about 2 s for a small request; `retry-after` said 2 s (RPM) and 21 s (TPM, for a 4,319-token request)	first poll 2 s after each 429 succeeded; buckets refill continuously (RPM 0.5/s, TPM 133 tokens/s)	high

Three findings changed the design. **The limits refill continuously.** reset-requests divided by requests used was exactly 86.40 s on every call, so the daily budget never resets; it trickles back. **Groq charges for the prompt plus the full max_tokens when it admits a request.** 13,208 characters with max_tokens 16 was "Requested 4319". My limiter now reserves the same way and refunds afterwards; before that, 16 of 72 drafts hit a surprise 429. **gpt-oss sometimes returns HTTP 200 with no text,** because hidden reasoning used up max_tokens. As the router, gpt-oss-20b did this 5.2% of the time (median 569 ms), so the router moved to qwen (220 ms, never empty).

The **stand-in** (standin/server.py) is an OpenAI-compatible streaming server that copies what the probe saw: the same buckets, headers and 429 bodies, and retry-after sized to the request. First-token time comes from a lognormal fitted to each model's probe p50/p95 (120b 695/936 ms, 20b 659/909, qwen 352/489), and output streams at the measured 633, 1,522 and 486 tokens/s. Switches add 503s, 8 s stalls, halved limits and an empty day. **Where it's least like Groq:** its answers are fake (it quotes the first source), so I only grade quality on real runs. Its slowdown under load (4% per in-flight request above 4) is my assumption, since Groq showed none at 16 concurrent. And it starts with full buckets; my real account was partly used.

3 Results

The real runs are real_steady, real_burst and all of Table 3; the rest used the stand-in. Percentiles are p50/p95/p99 by linear interpolation over each row's requests. With small samples, p99 is close to the maximum. First token counts every reply that released text, total time includes people who gave up, rate-limited includes passages-only replies, and peak queue counts requests beyond 12 running pipelines.

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
real_steady	real	14	173	4.8	1654 / 2505 / 2521	1654 / 2505 / 2521
real_burst	real	10	3	206.0	1983 / 2488 / 2644	1983 / 2488 / 2644
steady	stand-in	17	103	9.9	1616 / 2452 / 3243	1616 / 2452 / 3244
burst	stand-in	42	58	43.1	2 / 2665 / 2889	3 / 2665 / 2889
overload	stand-in	64	121	31.9	632 / 2645 / 3339	632 / 2645 / 3339
overload_naive_retry	stand-in	64	121	31.8	1468 / 1917 / 2795	1468 / 1917 / 2795
rpm_halved	stand-in	17	102	10.0	552 / 2087 / 2386	552 / 2087 / 2386
daily_exhausted	stand-in	10	57	10.5	1 / 13 / 18	1 / 13 / 18
chaos	stand-in	17	102	10.0	1933 / 3497 / 3752	1956 / 4066 / 4865

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
real_steady	100.0	0.0	0.0	0.00	0	2082	$0 free tier ($0.40 at list price)
real_burst	90.0	10.0	0.0	0.10	0	2019	$0 free tier ($0.39 at list price)
steady	100.0	0.0	0.0	0.00	0	2739	$0 free tier ($0.49 at list price)
burst	31.0	69.0	0.0	0.07	0	2012	$0 free tier ($0.36 at list price)
overload	82.8	17.2	0.0	0.00	0	1303	$0 free tier ($0.25 at list price)
overload_naive_retry	28.1	71.9	0.0	2.47	0	1958	$0 free tier ($0.36 at list price)
rpm_halved	100.0	0.0	0.0	0.00	0	1378	$0 free tier ($0.27 at list price)
daily_exhausted	0.0	100.0	0.0	0.20	0	0	$0 free tier ($0.00 at list price)
chaos	94.1	0.0	5.9	0.06	0	2409	$0 free tier ($0.44 at list price)

Answer quality on 18 questions (14 answerable, 4 not, three of them traps): the setting I chose got 14/14 right with citations and refused all 4 correctly, at 1,679 tokens per answer. Under real steady load: 11 of 14 right, 2 correctly refused, and 1 correct answer (q11) wrongly thrown out by the LLM verifier. **Numbers I couldn't have guessed:** the daily refill of one request every 86.4 s; qwen's hidden 7,000 input-tokens-a-minute limit, seen only in 429 bodies; and how badly a naive client does under the same overload: 240 calls and 204 429s, against 70 and one for mine.

4 Where the time goes

The slowest request in the chaos run (stand-in, 5% of calls stall for 8 s) is chaos-000541-0007, at 3,816 ms (scripts/explain_request.py):

Step	Model	Start to end (ms)	Kind
understand	qwen3.8-27b	0 to 337	working
retrieve	local BM25	338 to 338	working
draft	qwen3.8-27b (120b was busy)	339 to 791	working
verify	gpt-oss-20b	791 to 3,816	failed: no first token in 3 s, cut and skipped

It spent 0 ms waiting, 790 ms working and 3,025 ms on a failed attempt, all before the first token, because an answer only goes out once its checks are done. The pipeline saves a timing span for every model call and budget wait (data/results/detail.jsonl). Requests like this one led to two fixes. I cut the first-token timeout from 6 s to 3 s: the slowest healthy first token in 345 real calls was 2,240 ms, and with 20% stalls a 3 s cut gave 94.1% success against 76.5%. I also stopped retrying best-effort steps. Before that, the slowest chaos request took 8,073 ms, and 75% of it was retrying.

5 When a limit is hit

Per-minute limit: people wait at most 2 s, and only when their preferred model is about to free up. After that they get a weaker answer, either from another model right away or in lean mode if the drafting model is backed up (lean got 12/14 right against 14/14, at 874 tokens instead of 1,679). If nothing frees within 3 s they get the cited passages with no model call (status rate_limited) rather than a long wait. Nobody is dropped without a reply. In rpm_halved I halved the per-minute limits without telling the client. It read the new limit from the first reply's `x-ratelimit-limit-tokens` header, switched to lean (1,378 tokens per answer against 2,739 in steady) and finished every request with no 429s. **Per-day limit:** once the drafting model is below 15% of its daily budget, everyone gets lean answers, which makes the rest last about twice as long. At zero (daily_exhausted), everyone gets cited passages in under 20 ms; the client learned the day was empty from the first 429 body. The day refills one request every 86.4 s, so the reply says roughly when a full answer will be possible again.

Retrieval: quality versus cost

Setting (piece size, pieces fetched, re-rank yes or no)	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
medium(piece=1200c,k=5,rerank=no)	18	72.2	0.0	22.2	1781	7512 / 12048	7512 / 12048
small(piece=600c,k=8,rerank=no)	18	77.8	0.0	22.2	1679	8685 / 11324	8685 / 11324
large(piece=2400c,k=3,rerank=no)	18	72.2	0.0	22.2	1996	8184 / 13963	8184 / 13963
medium_rerank(piece=1200c,k=4,rerank=yes)	18	72.2	5.6	22.2	2692	11868 / 15151	11868 / 15151
small_lean(piece=600c,k=4,rerank=no)	18	66.7	11.1	22.2	874	729 / 7886	729 / 7886

Every setting drafted on gpt-oss-120b with the verifier off, so retrieval is the only thing that changes (16 of the first 72 drafts fell back to qwen because of the 429 issue above). The differences are one question each, which is noise at this size, so cost decided it: **small pieces (600 characters, 8 fetched, no re-rank)** were never worse and used the fewest tokens. Re-ranking cost 51% more tokens and bought nothing. Table 3's latencies are mostly waits for 120b's bucket (sequential runs pinned to one model); the drafting call itself has a median of 678 ms over 92 real calls. **When the limit gets close,** retrieval shrinks to 4 pieces with no router and no re-rank, which halves input tokens. The small_lean row shows what that costs in quality.

How this library will mislead a naive retriever

(Raw question, plain BM25, references included; scripts/retriever_traps.py.) 1. **Same term, wrong paper.** Ask "What word error rate does H-Net achieve on LibriSpeech?" and the top hit is Moshi 5.7, where the LibriSpeech WER is 4.7%. A naive pipeline answers 4.7% for H-Net. Mine refused in all five settings, even lean mode with no router: the prompt says to answer only from the sources, and they were clearly about Moshi. 2. **Vocabulary mismatch.** For "What is the frame rate of the Mimi codec?", the passage that says "12.5Hz" ranks 18th, out of reach of 5 or 8 pieces, because the paper doesn't say "frame rate" there. My q06 also asks for the bitrate, and "1.1kbps" is in the same passage, so it got answered every time; asked alone, it's a trap. The router's rewritten query is meant to catch this, but my test set doesn't prove that it does. 3. **Tables and numbers break in extraction.** Attention's Table 2 ends up under "5.4 Regularization", and its training cost of 2.3 x 10^19 FLOPs comes out as "2.3 · 1019". So "What did the big model cost in FLOPs?" can be answered off by 18 orders of magnitude. The drafting prompt warns about flattened exponents, but the citation still points at the wrong section.

6 Retries

Each step gets at most one retry, and never on the bucket that just returned a 429: it switches model. There's no retry after a daily-limit 429, and retries are capped at 10% of all attempts so they can't pile on load. Best-effort steps (verifier, re-rank) are never retried, and a call that returns no text counts as failed. **Retrying on its own makes things worse.** overload_naive_retry is a client with no limiter that retries the same model up to 3 times, on the same arrivals as overload. It made 240 calls, got 204 429s and succeeded 28.1% of the time. Mine made 70 calls, got one 429 and succeeded 82.8%. Every naive retry hit a bucket that was still empty.

7 What the person sees

Say it's 3pm and the daily budget is gone. Within 20 ms they see: "I can't write a full answer right now (our model capacity is used up), but these are the passages from the papers that match your question:", then up to three passages, each with paper, section, page and a 300-character quote, and finally "A full answer should be possible again in about 1 min." The estimate comes from the refill rate. They get evidence and an honest time, not a spinner.

8 What I'm unsure about, what I left out, what I learned

The assumption I'm least sure of is that limits are per model and independent. Every 429 names a model, and three walls hit at once didn't affect each other. But if Groq pooled limits per organisation, I'd have a third of the capacity I'm planning for, and the ladder would kick in three times sooner. Also untested: Groq at daily-limit zero, and more than 8 requests in flight. **I left two things out on purpose.** Embeddings and vector search would add another rate-limited dependency, and BM25 with a router already got 14/14. Streaming tokens before the check would save well under a second (a 250-token draft is quick on Groq), and holding the answer means I never have to take one back. **Next steps:** probe Groq at daily-limit zero and with more than 8 requests in flight; test whether limits are pooled per organisation; add a question that isolates the vocabulary-mismatch trap, to see whether the router's rewrite really fixes it; and find out why the verifier rejected q11 before it is allowed to block more answers. **What was new to me:** a rate limit can be a bucket that refills rather than a window that resets. Groq's "1,000 requests a day" is a bucket of 1,000 that gains one request every 86.4 s. That changes what a 429 means. It isn't "come back tomorrow", it's "come back once enough has dripped back in for a request your size". That's why a big request was told to wait 21 s while a tiny one got through 2 s later, and why the fix is to make the request smaller (lean mode) or send it to another bucket (another model), not to retry the same request.

Your feedback on Vaani

What stood out: the case studies use real volumes (69,990 calls placed, 305 hours of automated talk time), and the latency claim is a percentile ("~450ms p90") rather than an average. That's the right instinct, and it's exactly what this assignment is about. **What confused me:** what a "voice AI harness" is, how Teacher AI, Memories and Call screening fit together, and whether the page is meant for developers or enterprise buyers. **One thing I'd change:** the headline numbers are animated counters that read 0 until you scroll to them, so link previews, crawlers and AI summaries of the site see "0M+" and "<0ms". I'd put the real values in the HTML, and say next to the latency claim what's measured (first audio byte?) and where.

How I worked with the assistant

I used Claude Code (Claude Opus) for the whole build. My prompts are in prompts.md, in the order I sent them. I set the rules up front: measure Groq before designing anything and treat its docs as a hypothesis; never make anyone wait for budget that won't arrive, but degrade and say so; and every reported number must come from a file a script produced. Then I specified each milestone: what the probe had to answer and its cap (220 requests, about 120K tokens, because the key is shared), the limiter (refilling buckets per model, synced from headers, reserving at the admission charge), the pipeline and its degrade ladder, the stand-in's switches, the load scenarios with a naive-retry baseline, the trap questions, the retrieval comparison and the retry policy. For tuning I gave it a loop: find the slowest request, split its time into waiting, working and retrying, change one thing, run two seeds, keep the change only if the numbers improve, and log the decision. That loop produced the wait cap, the qwen router, the 3 s first-token timeout and no retries for best-effort steps, and overload success went from 51.6% in the first version (data/experiments/iteration0_standin.csv) to 82.8%. **It got things wrong along the way.** It first passed each simulated person's hidden patience to the pipeline as the deadline, which my load-test prompt had ruled out. Its citation check threw out correct answers because gpt-oss writes "[S1]" with narrow no-break spaces. And its token estimate under-counted Groq's admission charge, so 16 of 72 drafts hit a surprise 429. These came out when its output was checked against the measurements. **What's mine:** the design rules, the milestone plan, what to measure and how to tune. **What came from the assistant:** the code, running the probe and experiments, the first draft of the eval questions (which I had it show me before they were used), and the first draft of this write-up, which it then audited claim by claim against the data.